

ines

mopo

28th January 2026

Chapter 1

ines spec

<i>entity class name</i>	<i>parameter name</i>	<i>valid types</i>
constraint	constant	float
constraint	sense	
link	availability	float 1d_map time_series
link	availability_forecasts	1d_map 2d_map
link	capacity	float
link	constraint_link_count_coefficient	1d_map
link	efficiency	float 1d_map time_series
link	efficiency_forecasts	1d_map 2d_map
link	fixed_co2_emissions	float 1d_map
link	fixed_cost	float 1d_map
link	flow_max_cumulative	float 1d_map
link	flow_min_cumulative	float 1d_map
link	interest_rate	float 1d_map
link	investment_co2_emissions	float 1d_map
link	investment_cost	float 1d_map
link	investment_method	
link	investment_uses_integer	
link	is_non_synchronous	
link	lifetime	float 1d_map
link	links_existing	float 1d_map
link	links_fix_cumulative	float 1d_map
link	links_invest_fix_period	1d_map
link	links_invest_max_period	1d_map
link	links_invest_min_period	1d_map
link	links_max_cumulative	float 1d_map
link	links_min_cumulative	float 1d_map
link	links_retire_period	1d_map
link	operational_cost	float 1d_map time_series
link	operational_cost_forecasts	1d_map 2d_map
link	retirement_method	
link	salvage_value	float 1d_map
link	transfer_method	
node	co2_content	float 1d_map
node	commodity_limit	float 1d_map
node	commodity_price	float 1d_map time_series
node	commodity_price_forecasts	1d_map 2d_map
node	commodity_price_ladder	1d_map
node	constraint_storage_count_coefficient	1d_map
node	constraint_storage_state_coefficient	1d_map
node	flow_annual	float 1d_map
node	flow_profile	float 1d_map time_series
node	flow_profile_forecasts	1d_map 2d_map

value t

so

link.inve

bo

bo

link.retire

link.trans

Chapter 2

Formulation ines

The ines spec is disentangled here in the typical formulation of a linear program: preprocessing, indices, parameters, variables, objective and constraints. The formulation parameters and variables are formatted as $p_{indices}^{name}$ and $v_{indices}^{name}$ respectively. The names correspond directly to the input parameters in the ines spec, unless otherwise specified in the preprocessing section. The indices are always the most complex possible datastructure making the equations valid for all possible input values.

2.1 Preprocessing

2.1.1 sets

A *set* groups certain entities and gives them common parameters for common constraints.

2.1.2 Temporal structure

A lot of the preprocessing of the temporal structure is defined by the *solve_pattern*.

The *solve_mode* dictates whether the model uses a single time horizon or a rolling time horizon (single by default). Rolling time horizon means that the same model is used for different horizons and that the input of the next horizon depends on the results of the current horizon. This is not reflected in the model formulation but rather in an algorithm.

The time horizon of every solve pattern is described by a *start_time* and a *duration*. It is sufficient to only give a *duration* value as it starts from the start of the horizon(?) If the solve mode is a rolling horizon, in addition you define the *rolling_horizon* which is also a duration but is smaller than the *duration* of the solve pattern. Each roll will have a length equal to the rolling horizon and will roll as many times as needed to reach the duration of the solve pattern. You can also define a *rolling_jump* to have a gap between each roll. The ines spec currently does not allow for variable roll durations and jumps as the only accepted value is a duration value.

includes_solve_pattern allows for nested setups. For example, you can combine a rolling model hydro model with a rolling dispatch model. When the hydro model includes the dispatch model it means that first the hydro model will roll, then the dispatch model will roll as many times within the rolling horizon of the hydro model (assuming the rolling horizon of the dispatch model is smaller than the rolling horizon of the hydro model).

For rolling and nested patterns to communicate with each other, there are some parameters in the nodes, connections and units to set the method to bind them by various solves (i.e. Benders decomposition). However, the parameters for which this holds is not complete (e.g. a method value 'fix based on upper level').

The *time_resolution* indicates how the data should be sampled. For example, hourly data can be sampled as two hourly data. Different resolutions in different parts of the system are realised with a *set*. The *time_resolution_scope* indicates whether the data has different time resolutions defined by sets or not. The time resolution of the solve pattern then needs to have the smallest possible resolution to be able to cover all time steps in the *timeline* (see later).

period is meant for a larger time scale, e.g. an investment time scale. It is defined by a *start_time* and a *years_represented*. Typically expressed in years, but sometimes there are also quarterly investments that is less than a year. $p_{period}^{years_represented}$ is used to derive the time steps that belong to a period. The used periods also need to be defined in the solve pattern.

The *system* entity has a parameter called the *timeline* and holds all of the allowed datetimes in the model. A lot of the conversion tools expect this timeline to be there. The timeline is a timeseries with each datetime representing a timestep and each value representing the duration which is a float that is to be interpreted as hours (even though one could expect to use a map with datetimes and durations, however, most tools use hours and it would make the conversion tools more difficult to develop).

2.1.3 Stochastic structure

The stochastic structure is organised as an alternative parameter value, i.e. you can either provide a deterministic value (e.g. *profile_limit_lower*) or a forecast value (e.g. *profile_limit_lower_forecast*). The forecasts can either be independent or branching. The branching forecasts require an additional dimension (i.e. a distinction between analysis time and forecast time?). The *stochastic_scope* method parameter (in the solve pattern) determines which stochastic structure the model uses. *whole_model* implies that the entire model uses the same stochastic structure and every parameter needs to be stochastic if it can be. *single_set* implies that the entire model uses the same stochastic structure but not all parameters need to be stochastic. *multiple_sets* implies that different parts of the model use different stochastic structures.

2.1.4 Economic representation

The ines spec assumes that the distribution of annuities is flat, i.e. there is no lead time. M_{period} is the milestone year of the period; could be start of the simulation for all periods.

$$\begin{aligned}
annuity_{unit,period} &= \frac{p_{unit,period}^{interest_rate}}{1 - (1 + p_{unit,period}^{interest_rate})^{-p_{unit}^{lifetime}}} \\
discount_factor_{unit,period} &= \sum_{y=Y_{period}}^{Y_{period}+p_{unit}^{lifetime}} \frac{1}{1 - (1 + p_{unit,period}^{interest_rate})^{y-M_{period}}} \\
salvage_factor_{unit,period} &= \begin{cases} 0 & Y_{end} \geq Y_{period} + p_{unit}^{lifetime} \\ \sum_{y=Y_{end}}^{Y_{period}+p_{unit}^{lifetime}} \frac{1}{(1 + p_{unit,period}^{interest_rate})^{y-M_{period}}} & Y_{end} < Y_{period} + p_{unit}^{lifetime} \end{cases} \\
p_{unit,period}^{economic_investment} &= annuity_{unit,period} \cdot (discount_factor_{unit,period} - salvage_factor_{unit,period}) \\
p_{unit,period}^{economic_fixed} &= \begin{cases} \frac{p_{period}^{years_represented}}{1 - (1 + p_{unit,period}^{interest_rate})^{Y_{period}-M_{period}}} & p_{period}^{years_represented} < 1 \\ \sum_{y=Y_{period}}^{Y_{period}+p_{period}^{years_represented}} \frac{1}{1 - (1 + p_{unit,period}^{interest_rate})^{y-M_{period}}} & p_{period}^{years_represented} \geq 1 \end{cases} \\
p_{unit,period}^{economic_other} &= \begin{cases} \frac{1}{1 - (1 + p_{unit,period}^{interest_rate})^{Y_{period}-M_{period}}} & p_{period}^{years_represented} < 1 \\ \sum_{y=Y_{period}}^{Y_{period}+p_{period}^{years_represented}} \frac{1}{1 - (1 + p_{unit,period}^{interest_rate})^{y-M_{period}}} & p_{period}^{years_represented} \geq 1 \end{cases}
\end{aligned}$$

The discount rate and the interest rate need to be specified for the tools to work.

2.1.5 units, nodes, links and flows

Units and links operate between nodes. A node has a *node_type* which indicates how the node is balanced. *balance* implies that all incoming and outgoing flows need to equal each other. *commodity* implies the use of a commodity to deal with the imbalance of flows in the node. A typical configuration is a 'source' node with a *commodity* node type connected to a unit (through a *node_to_unit* entity) and the unit connected to a 'demand' node with a *balanced* node type (through a *unit_to_node* entity). The 'demand' node would then also have a parameter *demand* which denotes the demand of the node in every time step.

The efficiency etc. is defined at the unit level, whereas the capacity is defined at the flow level (although the flow can also define an efficiency that is more flexible, similar to SpineOpt). The capacity needs to be defined for every flow. The efficiency needs to be defined for every unit. The capacity is per unit as it is multiplied with the number of existing units. The direction of the flow is handled by *node_link_node*, on one hand in the transfer method in *link* and on the other hand with multiplier from *constraint_flow*. To deal with multiple profiles for a unit in a multiyear setup, you can define a unit with the same parameters but different profiles (though in some setups you would require a map with a period and a time, which is currently not supported).

unit_conversion_method provides a few conversion methods including *partial_load_efficiency* and *piecewise_linear* as depicted in the figure below.

To define a cyclic condition for storage in a node, you set the *storage_binding_method* to leap over a period or solve. You can also fix the initial state of the storage with the *storage_fix_state_method*.

Across the entities there are some scaling parameters (methods) that can be used to, e.g., scale the flow of a partial solve to a year.

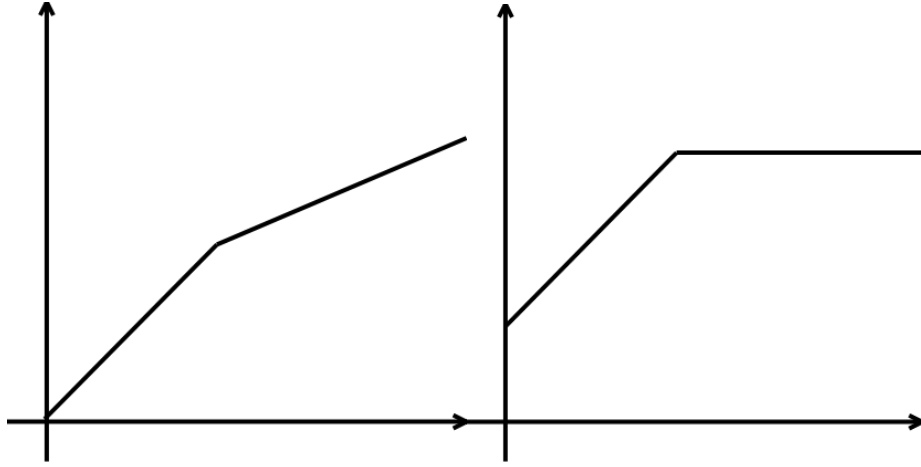


Figure 2.1: Visual depiction of *unit_conversion_method*: piece wise linear (left) and partial load efficiency (right)

Some entities also have methods for investments. When it is cumulative, it describes over multiple periods what is the total allowed capacity. Which can also be set for a set.

To set up a user constraint you define a *constraint* with the *sense* and *constant*. You can then link a *unit_to_node* entity to this constraint through the *constraint_flow_coefficient*.

2.2 Indices

set groups of units, connections, etc.

set_{unit} set that a unit belongs to

set_{connection} set that a connection belongs to

set_{node} set that a node belongs to

period investment periods in the system

time time steps (always continuous within period or rolling horizon, weight is used for representative days)

time_{set} time steps in set

period_{time} period corresponding to time

unit units in the system

unit_to_node unit to nodes in the system

unit_{unit_to_node} unit in the unit_to_node entity

node_to_unit unit to nodes in the system

unit_{node_to_unit} unit in the unit_to_node entity

2.3 Parameters

The parameters are formatted as $p_{sets}^{parameter_name}$ with the parameter name from the table below and the set based on the maximum of the valid types.

`node__to_unit` has the same parameters as `unit__to_node` so that is not repeated here

$p_{unit}^{interest_rate}$	discount rate related to a unit
$p_{unit,period,time}^{startup_cost}$	startup cost
$p_{unit,period,time}^{shutdown_cost}$	shutdown cost
$p_{unit,period,time}^{online_cost}$	online cost
$p_{unit_to_node,period}^{investment_cost}$	investment cost of a unit flow in a certain period
$p_{unit_to_node,period}^{fixed_cost}$	FOM cost of a unit flow
$p_{un,p,t}^{other_operational_cost}$	VOM cost of a unit flow
$p_{unit_to_node,period}^{ramp_cost}$	the ramp cost is specifically described to be the cost of ramping up (not ramping down)

2.4 Variables

According to the ines-output spec, for a `node__to_unit`, there is a capacity, a flow per period and a flow per time step.

`node__to_unit` is the same as `unit__to_node` so it is not repeated here.

$v_{un,p,t}^{commit}$	committed units
$v_{un,p,t}^{commit_up}$	units that started up
$v_{un,p,t}^{commit_down}$	units that shut down
$v_{unit_to_node,period}^{investment}$	additional capacity of installed <code>unit__to_node</code> in a period
$v_{unit_to_node,period}^{capacity}$	capacity of <code>unit__to_node</code> in each period
$v_{unit_to_node,period,time}^{flow}$	flow of <code>unit__to_node</code> in each time in each period
$v_{unit_to_node,period,time}^{rampup}$	ramp up of <code>unit__to_node</code> in each time in each period

2.5 Objective

min

$$\begin{aligned}
& + \sum_{u \in \text{unit}} \sum_{p \in \text{period}} p_{u,p}^{\text{economic_other}} \cdot \sum_{t \in \text{time}_p} \Delta_t \cdot p_{un,p}^{\text{online_cost}} \cdot v_{un,p,t}^{\text{commit}} \\
& + \sum_{u \in \text{unit}} \sum_{p \in \text{period}} p_{u,p}^{\text{economic_other}} \cdot \sum_{t \in \text{time}_p \notin 1} p_{un,p}^{\text{startup_cost}} \cdot v_{un,p,t}^{\text{commit_up}} \\
& + \sum_{u \in \text{unit}} \sum_{p \in \text{period}} p_{u,p}^{\text{economic_other}} \cdot \sum_{t \in \text{time}_p \notin 1} p_{un,p}^{\text{shutdown_cost}} \cdot v_{un,p,t}^{\text{commit_down}} \\
& + \sum_{un \in \text{unit_to_node}} \sum_{p \in \text{period}} p_{unit[un],p}^{\text{economic_investment}} \cdot p_{un,p}^{\text{investment_cost}} \cdot v_{un,p}^{\text{investment}} \\
& + \sum_{un \in \text{unit_to_node}} \sum_{p \in \text{period}} p_{unit[un],p}^{\text{economic_fixed}} \cdot p_{un,p}^{\text{fixed_cost}} \cdot v_{un,p}^{\text{capacity}} \\
& + \sum_{un \in \text{unit_to_node}} \sum_{p \in \text{period}} p_{unit[un],p}^{\text{economic_other}} \cdot \sum_{t \in \text{time}_p} \Delta_t \cdot p_{un,p,t}^{\text{other_operational_cost}} \cdot v_{un,p,t}^{\text{flow}} \\
& + \sum_{un \in \text{unit_to_node}} \sum_{p \in \text{period}} p_{unit[un],p}^{\text{economic_other}} \cdot \sum_{t \in \text{time}_p \notin 1} p_{un,p}^{\text{ramp_cost}} \cdot v_{un,p,t}^{\text{ramp_up}} \\
& + \sum_{nu \in \text{node_to_unit}} \sum_{p \in \text{period}} p_{unit[nu],p}^{\text{economic_investment}} \cdot p_{nu,p}^{\text{investment_cost}} \cdot v_{nu,p}^{\text{investment}} \\
& + \sum_{nu \in \text{node_to_unit}} \sum_{p \in \text{period}} p_{unit[nu],p}^{\text{economic_fixed}} \cdot p_{nu,p}^{\text{fixed_cost}} \cdot v_{nu,p}^{\text{capacity}} \\
& + \sum_{nu \in \text{node_to_unit}} \sum_{p \in \text{period}} p_{unit[nu],p}^{\text{economic_other}} \cdot \sum_{t \in \text{time}_p} \Delta_t \cdot p_{nu,p,t}^{\text{other_operational_cost}} \cdot v_{nu,p,t}^{\text{flow}} \\
& + \sum_{nu \in \text{node_to_unit}} \sum_{p \in \text{period}} p_{unit[nu],p}^{\text{economic_other}} \cdot \sum_{t \in \text{time}_p \notin 1} p_{nu,p,t}^{\text{ramp_cost}} \cdot v_{nu,p,t}^{\text{ramp_up}}
\end{aligned}$$

2.6 Reserves

$\forall n \in \text{node_reserve} :$

$$\begin{aligned}
& \sum_{u \in n} r_{u,t}^+ + \sum_{u \in su \cup n} r_{u,t}^{c,+} + r_{u,t}^{d,+} + \sum_{u \in dlu \cup n} r_{u,t}^{l,+} + \sum_{u \in du \cup n} r_{u,t}^{ll} \\
& - \sum_{u \in du \cup n} F^{\text{dres}} \cdot P_{u,Y_t}^{\text{peak}} - \sum_{u \in ru \cup n} F^{\text{res}} \cdot c_{u,Y_t} = 0 \\
& \sum_{u \in n} r_{u,t}^- + \sum_{u \in su \cup n} r_{u,t}^{c,-} + r_{u,t}^{d,-} + \sum_{u \in dlu \cup n} r_{u,t}^{l,-} \\
& - \sum_{u \in du \cup n} F^{\text{dres}} \cdot P_{u,Y_t}^{\text{peak}} - \sum_{u \in ru \cup n} F^{\text{res}} \cdot c_{u,Y_t} = 0
\end{aligned}$$

2.7 Unit Constraints

$$\begin{aligned}v_{un,p,t}^{ramp-up} &> 0 \\v_{un,p,t}^{ramp-up} &> v_{un,p,t}^{flow} - v_{un,p,t-1}^{flow} \\v_{un,p,t}^{ramp-down} &> 0 \\v_{un,p,t}^{ramp-down} &> v_{un,p,t-1}^{flow} - v_{un,p,t}^{flow}\end{aligned}$$

Chapter 3

Certification process

Whereas the ines spec concerns the parameters of the imaginary ines model, Ines certify considers the variables of that same model. To make that more clear, we write down the equations in [Appendix 2](#).

The certification process describes a few setups in the ines spec and holds the result in the ines certify. The workflow then consists of the conversion from the ines spec to different tools and later the conversion to ines certify with a check of the results. The latter is done manually for now but can be automated in the future.

The certification setups consist of equation tests, method test and system tests.

equation tests are a test for each equation in the ines model to test whether that feature is available in the respective model

method tests are tests that cover multiple equations to show off a certain feature

system tests are more comprehensible, similar to IEEE networks

Though, most tests use multiple equations so the difference is small anyway. Therefore, we do not make a distinction between these categories in the following tests.

3.1 network = copper plate

Energy balance where the demand equals the supply for a single time step. (Maybe two units where one is cheaper than the other to also test the minimum cost objective?)

The current implementation is also an investment model as the conversion tools so far assume investment models. Additionally, and for a similar reason, there is a fixed time resolution for a year.

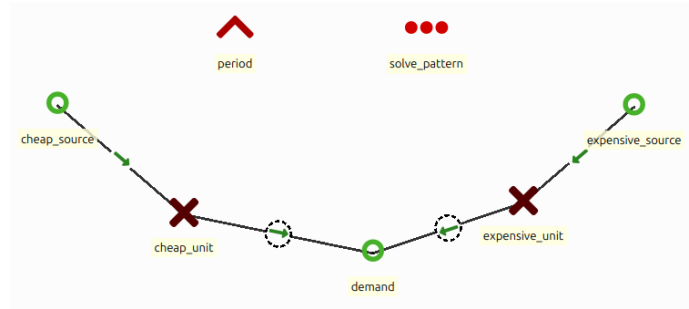


Figure 3.1: Network = copper plate

3.2 network = radial

Two supply units, each connected with a directional line to the demand. The cheaper unit has the direction from the demand to the supply so the setup has no choice but to choose the more expensive unit.

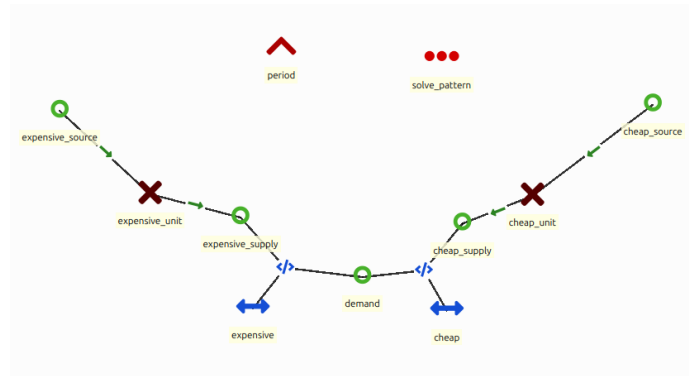


Figure 3.2: Network = radial

Currently does not work for flextool, uses the penalty instead. That is because the links don't have parameters yet.

3.3 network = mesh

Triangular network with the two supply units and demand unit. The cheapest unit should be chosen again.

3.4 resolution = fixed

Same setup but with a time series for all units with the same resolution.

3.5 resolution = variable

Same setup but the time series has a variable resolution.

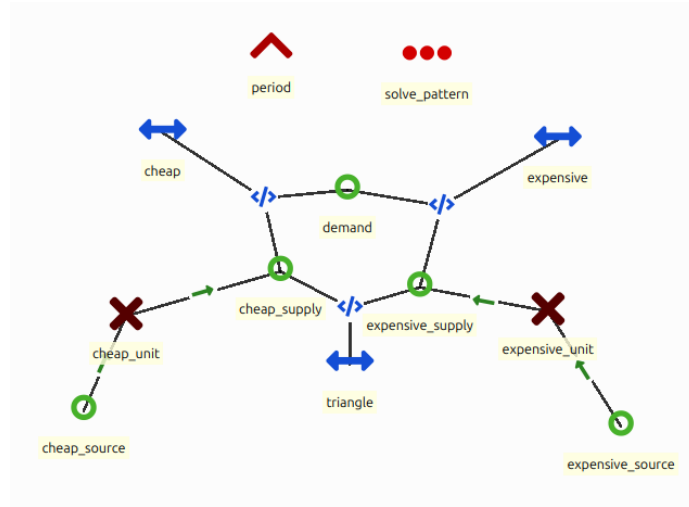


Figure 3.3: Network = mesh

3.6 resolution = flexible

Same setup but the two units have different resolutions (1h for expensive unit and 2h for cheap unit).

3.7 stochastic

Meshed network, fixed resolution, expensive and cheap unit. The units have the same stochastics.

3.8 stochastic = flexible

Same setup but the units have different stochastics.

3.9 CO2 cap

Meshed network, fixed resolution, expensive and cheap unit. The cheap unit produces CO2 and is limited by the cap.

3.10 reserves

Meshed network, fixed resolution, expensive and cheap unit. The expensive unit is able to provide reserves but not enough to entirely avoid load shedding.

3.11 cogeneration

Meshed network, fixed resolution, two supply units and two demand units but one supply is cogeneration and one demand is for heat. The setup is heat

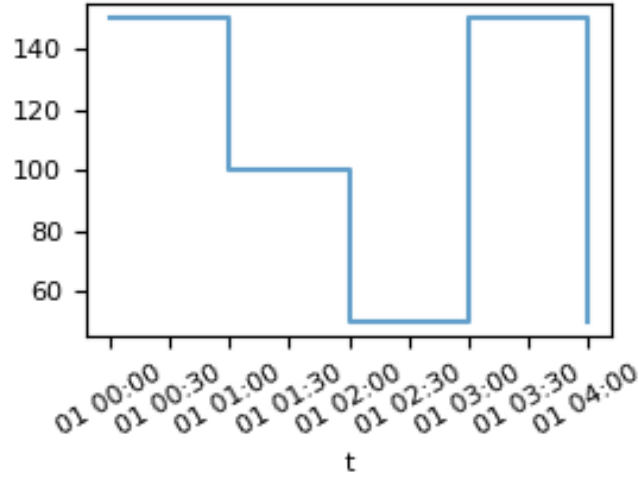


Figure 3.4: resolution = fixed

driven as only the electricity grid has two supply units.

3.12 ramping

Meshed network, fixed resolution, expensive and cheap unit. The cheap unit is not able to ramp. The demand profile is either an elevated sine or piecewise linear \-/.

3.13 flow = fixed

Meshed network, fixed resolution, expensive and cheap unit. The expensive unit is forced to produce a certain flow.

3.14 flow = limits

Meshed network, fixed resolution, expensive and cheap unit. The expensive unit is forced to produce within certain limits. This test also represents curtailment.

3.15 storage = intra

Meshed network, fixed resolution, one storage unit and one supply unit. The supply unit has a fixed output that does not correspond to the demand. The storage has to be used to be feasible.

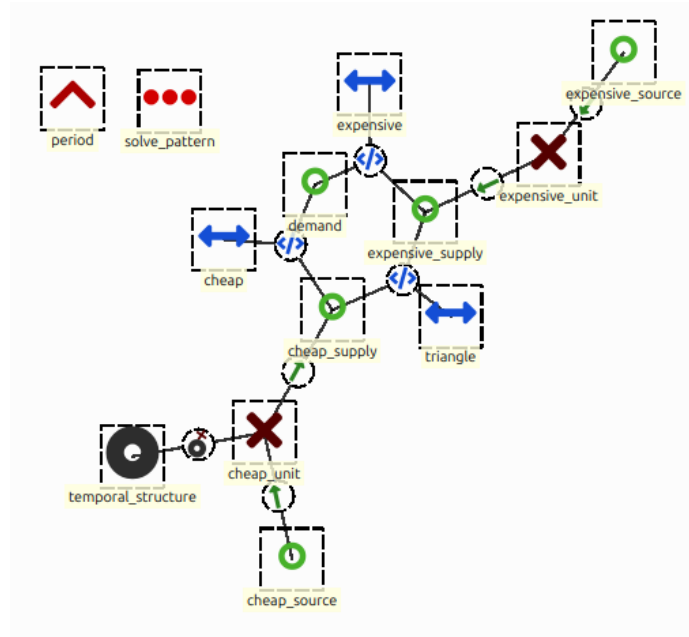


Figure 3.5: resolution = flexible

3.16 storage = inter

Same setup but there are multiple periods.

3.17 Part load efficiency

Meshed network, fixed resolution, expensive and cheap unit. The partload efficiency of the cheap unit makes it that the overall cost is higher than the expensive unit.

3.18 MUT

Meshed network, fixed resolution, expensive storage and cheap unit. By setting the minimum production equal to the maximal production the output of the unit becomes fixed. By setting the minimum up time (*MUT*) equal to the lifetime of the unit, whenever the unit delivers an output, that output will need to be maintained for the remainder of the simulation. That means that there will appear horizontal lines for each level of required output. To make sure that the system works, a storage unit is used to get rid of the excess power. The storage unit is very expensive such that it would be avoided if it would not be needed.

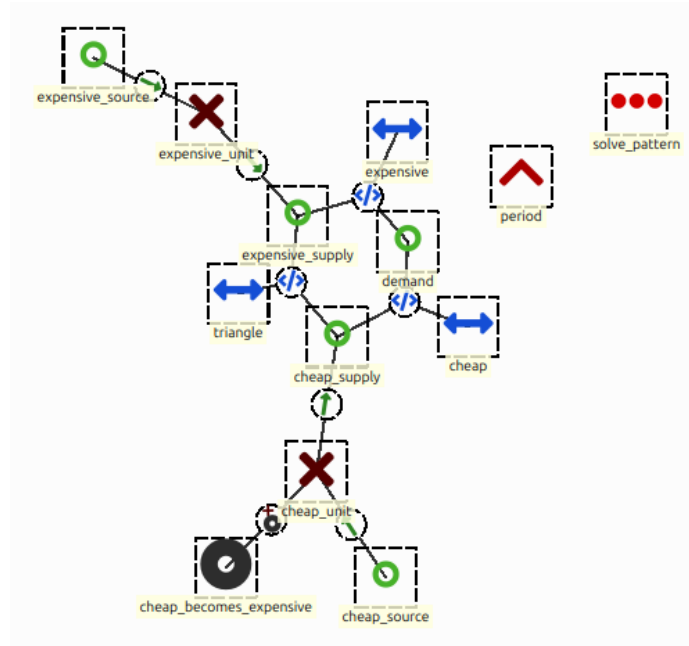


Figure 3.6: stochastic

3.19 MDT

Same setup, but there is a moment where the demand is zero. Normally, the plant would shut down, but due to the MDT it needs to remain on.

3.20 economic representation

Meshed network, fixed resolution, expensive and cheap unit. The numbers will be different from other setups due to the economic representation.

3.21 investment = fixed

Meshed network, fixed resolution, expensive and cheap unit. Expensive unit is already invested in.

3.22 investment = cumulative

Same setup but a cumulative cap

3.23 investment = multi-year

Same setup but multiple periods.

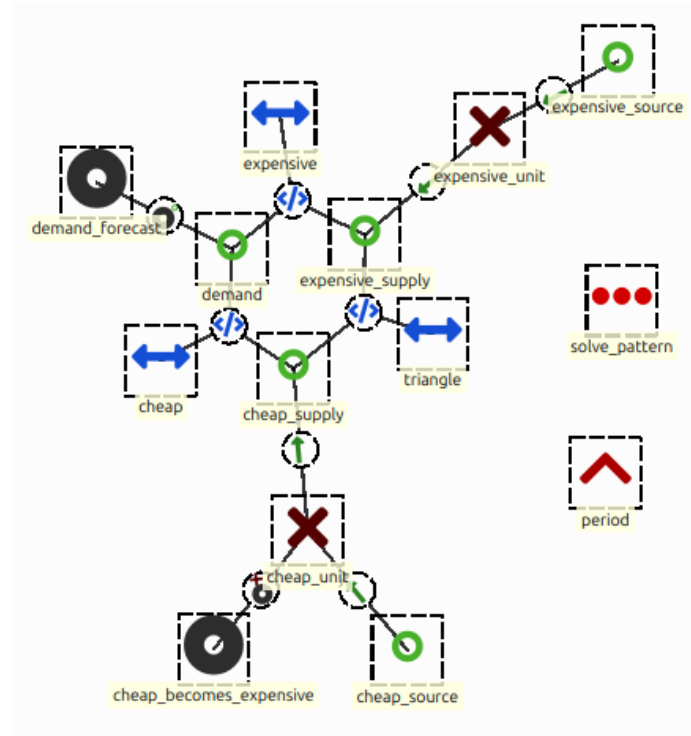


Figure 3.7: stochastic = flexible

3.24 retirement = fixed

Same setup Expensive unit is already invested in but is sunset in the second period.

3.25 retirement = economic

Same setup but the retirement is not fixed.

3.26 rolling horizon

Same setup but the periods are separated.

3.27 decomposition

Same setup but the investment and operation are separated from each other.

3.28 mini europe

A miniature version of europe with very limited temporal aspects (a day = 6 hours, a season is 1 day). Each country has a demand but only its most

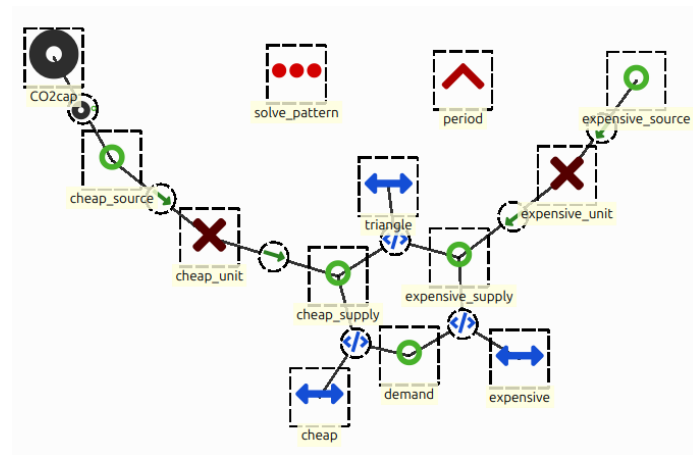


Figure 3.8: CO2cap

prominent technologies.

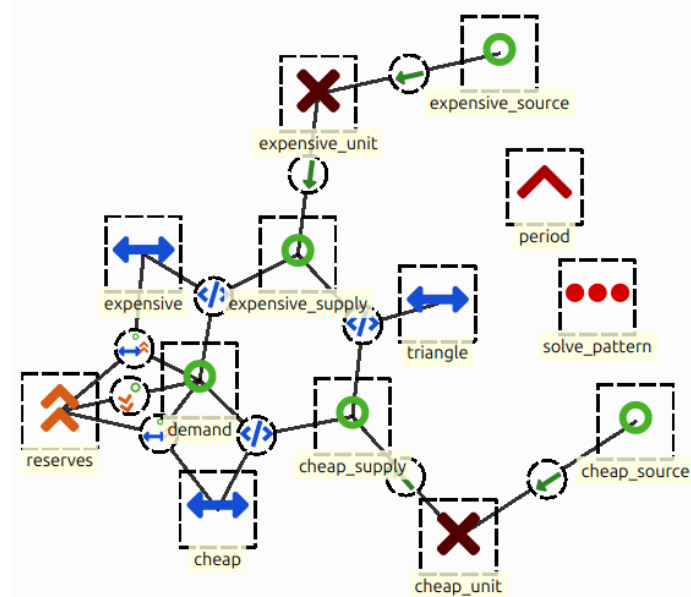


Figure 3.9: reserves

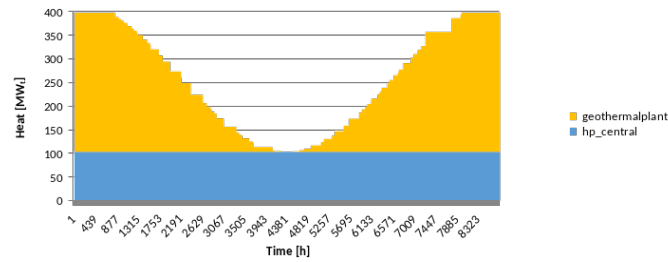


Figure 3.10: Test for ramping. There are two technologies (a geothermal plant and a heat pump) to supply the heat demand. The geothermal plant is expensive but is set to have a fast ramp up. The heat pump is cheap but is set to have a very slow ramp up. The system would like to choose the heat pump at all times but the heat pump can only be used as a base supply because of the ramp up times.

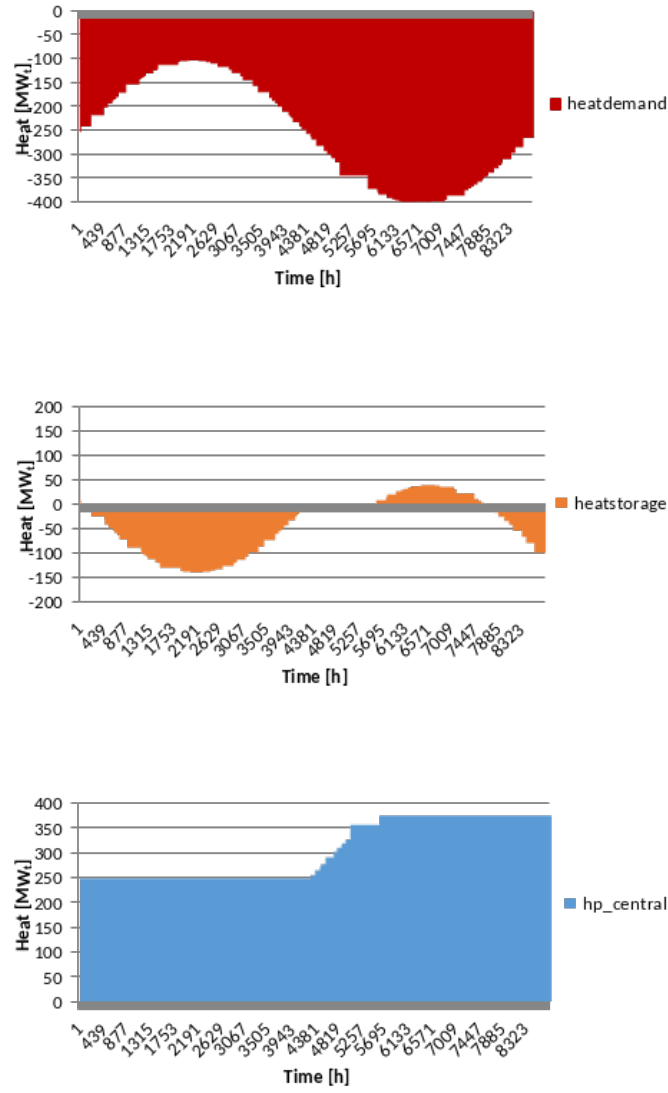


Figure 3.11: Test for commitment. There is one unit to supply heat but the unit can only produce at full committed capacity and the minimum up time of that unit is set to the length of the simulation. That means that once the unit starts producing at a certain capacity it cannot go lower anymore. A storage unit is used to compensate for that inflexible behaviour.